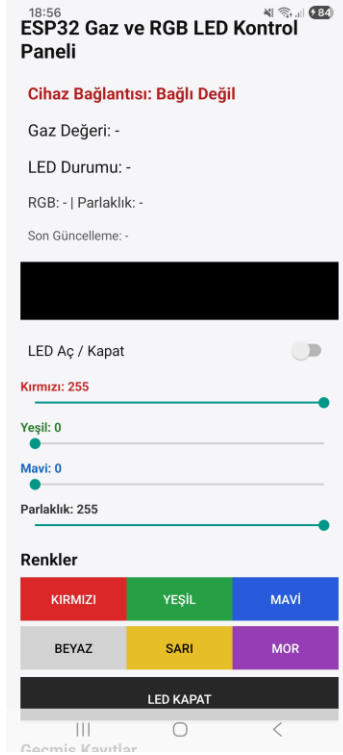


ESP32 Gaz İzleme ve RGB LED Kontrol Projesi

Android Mobil Uygulama, BLE, Wi-Fi, MQTT, Spring Boot Backend ve SQLite Tabanlı
Türkçe Proje Dokümantasyonu

Alan	Açıklama
Proje Türü	IoT + Mobil Uygulama + Backend
Donanımlar	ESP32-S3, MQ-2 gaz sensörü, WS2812 RGB LED
Yazılım Bileşenleri	Arduino C++, Android Java, Spring Boot, MQTT, WebSocket, SQLite
Doküman Dili	Türkçe



İÇİNDEKİLER

1. Proje Özeti	1
2. Amaç ve Kapsam	1
3. Kullanılan Teknolojiler	1
4. Repository Yapısı	2
5. Genel Sistem Mimarisi	3
5.1 BLE mimarisi	3
5.2 Wi-Fi/MQTT/backend mimarisi	3
6. Donanım Bağlantıları	4
7. BLE Sürümü	5
7.1 Esp32 BLE tarafı	5
7.2 Android BLE tarafı	6
8. Wi-Fi/MQTT Sürümü	7
8.1 ESP32 Wi-Fi/MQTT tarafı	7
9. Spring Boot Backend	8
9.1 Backend bağımlılıkları	8
9.2 Backend sınıfları	8
9.3 REST ve WebSocket endpointleri	8
9.4 Cihaz bağlantı durumu	9
10. Android Uygulama Yapısı	9
10.1 Wi-Fi Android uygulaması	9
10.2 Android UI davranışı	9
11. JSON Veri Formatları	10
12. Veritabanı Tasarımı	10
13. Kurulum ve Çalıştırma	11
13.1 ESP32 BLE sürümü	11
13.2 Wi-Fi/MQTT sürümü	11
14. Test Senaryoları	12
15. Hata Yönetimi ve Öneriler	12
15.1 Olası sorunlar	12
15.2 İyileştirme önerileri	13
16. Sonuç	13

1. Proje Özeti

Bu proje, ESP32-S3 geliştirme kartı üzerinde bağlı MQ-2 gaz sensöründen analog gaz değeri okumayı, kart üzerindeki veya harici WS2812 RGB LED'i mobil uygulama üzerinden kontrol etmeyi ve sistem durumunu Android tarafında gerçek zamanlı göstermeyi amaçlayan bir IoT projesidir. Repository iki farklı haberleşme yaklaşımını içerir: BLE tabanlı doğrudan haberleşme ve Wi-Fi/MQTT tabanlı backend üzerinden haberleşme.

Sürüm	Haberleşme Modeli	Temel Mantık
BLE sürümü	Android ↔ ESP32 BLE GATT	Telefon doğrudan ESP32'ye bağlanır, ESP32 state bilgisini notification olarak gönderir, telefon kontrol komutunu characteristic write ile yollar.
Wi-Fi/MQTT sürümü	ESP32 ↔ MQTT ↔ Spring Boot ↔ Android	ESP32 telemetriyi MQTT ile backend'e gönderir; backend Android'e WebSocket ile anlık state yayınlar, geçmiş kayıtları SQLite'da tutar.

2. Amaç ve Kapsam

- MQ-2 gaz sensöründen okunan değeri Android arayüzünde anlık göstermek.
- WS2812 RGB LED'in açık/kapalı durumunu, kırmızı-yeşil-mavi renk bileşenlerini ve parlaklığını mobil uygulamadan kontrol etmek.
- BLE ve Wi-Fi tabanlı haberleşme yöntemlerini aynı proje mantığı üzerinde karşılaştırılabilir şekilde göstermek.
- Wi-Fi sürümünde MQTT, WebSocket, REST API ve SQLite ile daha genişletilebilir bir IoT mimarisi kurmak.
- Cihaz bağlantı durumunu takip ederek cihaz bağlı değilken mobil arayüzde bunu açıkça göstermek.
- Belirli aralıklarla telemetri geçmişi tutarak son kayıtların Android uygulamada görüntülenmesini sağlamak.

3. Kullanılan Teknolojiler

Kategori	Teknoloji / Bileşen	Projede Kullanım Amacı
Donanım	ESP32-S3	Sensör okuma, BLE/Wi-Fi haberleşmesi ve RGB LED kontrolü.
Donanım	MQ-2 gaz sensörü	Analog gaz değerini okumak için kullanılır.
Donanım	WS2812 RGB LED	Mobil uygulamadan renk, parlaklık ve açık/kapalı kontrolü.
Gömülü yazılım	Arduino C++	ESP32 kodlarının Arduino IDE ile yazılması ve yüklenmesi.
Mobil	Android Java	Mobil arayüz, BLE GATT bağlantısı, WebSocket istemcisi ve HTTP geçmiş isteği.
Backend	Spring Boot	MQTT mesajlarını işleme, WebSocket yayını, REST API ve SQLite işlemleri.
Mesajlaşma	MQTT / Eclipse Paho / PubSubClient	ESP32 ile backend arasında telemetri ve kontrol mesajlaşması.
Gerçek zamanlı iletişim	WebSocket / OkHttp	Backend'den Android'e anlık cihaz state aktarımı.
Veritabanı	SQLite + JdbcTemplate	Telemetri geçmişini yerel dosya tabanlı veritabanında saklama.

4. Repository Yapısı

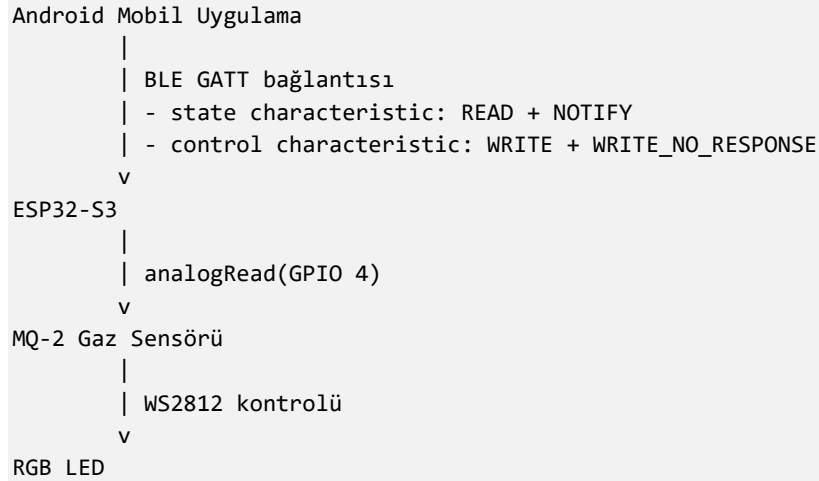
Repository içeriği incelendiğinde proje dosyaları pratik olarak aşağıdaki ana klasörlere ayrılmıştır:

```
ESP32-Gas-Monitoring-RGB-Control-onAndroid/
|
|-- README.md
|-- .gitignore
|
|-- ESP32_Mobile_app_bluetooth/
|   |-- esp32_bluetooth/
|   |   |-- esp32_bluetooth.ino
|   |
|   |-- android_mobile_app/
|   |   |-- app/src/main/java/com/batuhan/esp32bluetoothmobileapp/MainActivity.java
|   |   |-- app/src/main/AndroidManifest.xml
|   |
|   |-- ESP32_Mobile_app_wifi/
|   |   |-- esp32_emb_src/
|   |   |   |-- esp32_emb_src.ino
|   |   |
|   |   |-- ESP32_Mobile/
|   |   |   |-- app/src/main/java/com/batuhan/esp32_mobile/MainActivity.java
|   |   |   |-- app/src/main/java/com/batuhan/esp32_mobile/config/AppConfig.java
|   |   |   |-- app/src/main/java/com/batuhan/esp32_mobile/network/
|   |   |   |-- app/src/main/java/com/batuhan/esp32_mobile/model/
|   |   |   |-- app/src/main/java/com/batuhan/esp32_mobile/util/
|   |   |
|   |   |-- esp32-backend/
|   |   |   |-- pom.xml
|   |   |   |-- src/main/resources/application.properties
|   |   |   |-- src/main/java/com/batuhan/esp32_backend/controller/
|   |   |   |-- src/main/java/com/batuhan/esp32_backend/service/
|   |   |   |-- src/main/java/com/batuhan/esp32_backend/mqtt/
|   |   |   |-- src/main/java/com/batuhan/esp32_backend/websocket/
|   |   |   |-- src/main/java/com/batuhan/esp32_backend/config/
```

5. Genel Sistem Mimarisi

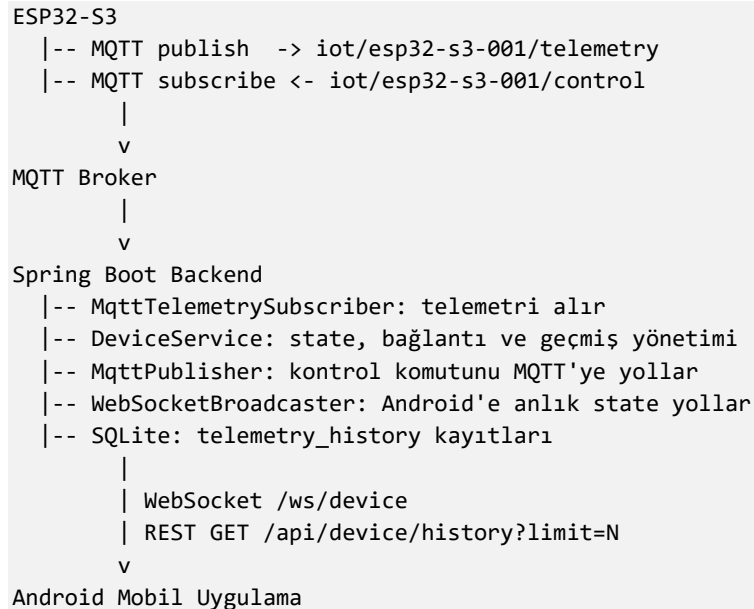
5.1 BLE mimarisi

BLE sürümünde backend yoktur. Android uygulama doğrudan ESP32'nin BLE servisine bağlanır. ESP32 bir GATT server gibi davranır; Android ise GATT client olarak bağlanır.



5.2 Wi-Fi/MQTT/backend mimarisi

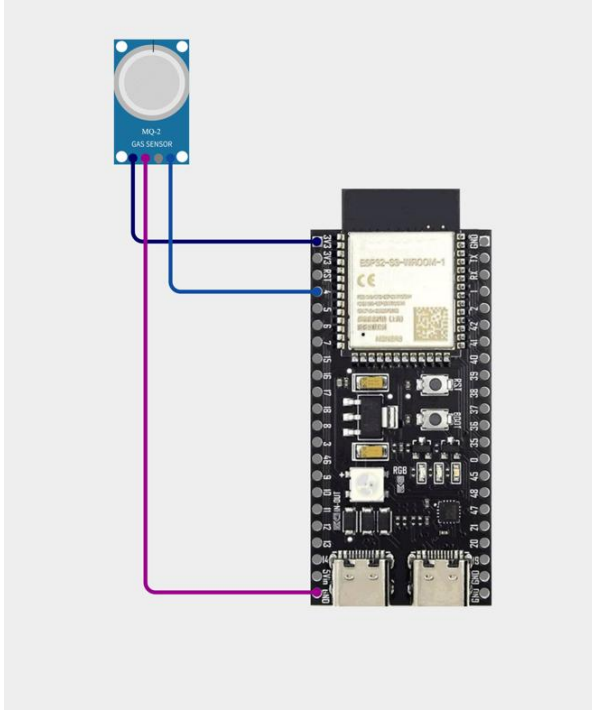
Wi-Fi sürümünde ESP32 doğrudan Android'e bağlanmaz. ESP32, MQTT broker üzerinden Spring Boot backend ile konuşur. Backend telemetriyi işler, SQLite veritabanına belirli aralıklarla kaydeder ve Android uygulamaya WebSocket üzerinden anlık state yayını yapar.



6. Donanım Bağlantıları

Bileşen	ESP32 Bağlantısı	Açıklama
MQ-2 AO / Analog çıkış	GPIO 4	Kodda GAS_PIN = 4 olarak tanımlanmış. Gaz değeri analogRead ile okunur.
MQ-2 VCC	5V veya modül gereksinimine göre 3.3V/5V	Kullanılan MQ-2 modülünün çalışma voltajına göre bağlanmalıdır.
MQ-2 GND	GND	Ortak toprak bağlantısı gerekir.
WS2812 DIN	GPIO 48	Kodda WS2812_PIN = 48 olarak tanımlanmış.
WS2812 VCC	3.3V/5V	Kart üzerindeki LED ise ek bağlantı gerekmeyebilir; harici LED’de besleme önemlidir.
WS2812 GND	GND	LED ve ESP32 aynı GND hattını paylaşmalıdır.

Not: GPIO 48 genellikle ESP32-S3 kartlarda kart üzerindeki WS2812/NeoPixel LED için kullanılır. Farklı ESP32 kartlarında pin değişebilir; kart pinout şemasına göre kontrol edilmelidir.



Donanım bağlantı şeması

7. BLE Sürümü

7.1 ESP32 BLE tarafı

ESP32 BLE kodu BLEDevice, BLEServer, BLEUtils, BLE2902, ArduinoJson ve Adafruit_NeoPixel kütüphanelerini kullanır. Cihaz adı ESP32-GAS-LED olarak tanımlanmıştır. Kodda tek bir BLE service ve iki characteristic kullanılır.

Alan	Değer / Görev
DEVICE_NAME	ESP32-GAS-LED
SERVICE_UUID	12345678-1234-1234-1234-1234567890ab
GAS_UUID	State characteristic. READ + NOTIFY özelliği ile gaz, bağlantı ve LED durumunu gönderir.
CONTROL_UUID	Control characteristic. Android'den gelen LED/RGB/parlaklık komutlarını WRITE ile alır.
SEND_INTERVAL_MS	200 ms. ESP32 state bilgisini yaklaşık 200 ms aralıklarla hazırlar ve bağlıysa notify eder.

ESP32'nin gönderdiği state mesajı kısa JSON anahtarlarıyla oluşturulur:

```
{
  "c": true,
  "g": 1234,
  "l": true,
  "r": 255,
  "gr": 0,
  "b": 0,
  "br": 255
}
```

JSON Alanı	Anlamı
c	Telefon/istemci bağlantı durumu
g	MQ-2 gaz sensörü analog değeri
l	LED açık mı kapalı mı
r	Kırmızı renk değeri, 0-255
gr	Yeşil renk değeri, 0-255
b	Mavi renk değeri, 0-255
br	Parlaklık değeri, 0-255

Not: Kod hem kısa anahtarları hem de ledOn, red, green, blue, brightness gibi uzun anahtarları okuyabilecek şekilde yazılmıştır. Bu, mobil taraf değişse bile ESP32'nin daha esnek çalışmasını sağlar.

7.2 Android BLE tarafı

Android BLE uygulaması MainActivity içinde BluetoothAdapter, BluetoothGatt, BluetoothGattCallback, characteristic ve descriptor işlemlerini yönetir. Uygulama belirlenen ESP32 MAC adresine bağlanır, servisleri keşfeder, notification açar ve gelen JSON state mesajlarını arayüze yansıtır.

Bileşen / Değişken	Görev
TARGET_MAC_ADDRESS	Bağlanılacak ESP32 cihazının MAC adresi. Kendi ESP32 adresiyle değiştirilmelidir.
BluetoothGattCallback	Bağlantı durumu, MTU sonucu, servis keşfi ve notification mesajlarını yakalar.
CCCD_UUID	BLE notification aktif etmek için kullanılan standart descriptor UUID'si.
REQUESTED_MTU = 256	BLE paket boyutunu büyötmeye çalışır; JSON mesajların daha rahat taşınmasını sağlar.
CONTROL_DEBOUNCE_MS = 100	SeekBar hızlı değişirken her milisaniyede komut yollamak yerine kısa gecikmeyle son komutu yollar.
LOCAL_CONTROL_PROTECT_MS = 700	Kullanıcı yeni kontrol yapmışsa ESP32'den gelen eski state'in UI kontrol değerlerini ezmesini engeller.

BLE Android akışı özetle şöyledir:

1. Uygulama açılır ve gerekli Bluetooth izinleri kontrol edilir.
2. Bluetooth açık ve izinler tamamsa TARGET_MAC_ADDRESS ile ESP32'ye bağlanılır.
3. Bağlantı kurulduğunda MTU isteği yapılır ve servis keşfi başlatılır.
4. GAS_UUID characteristic'i için notification aktif edilir.
5. ESP32'den gelen JSON state mesajları parse edilir ve gaz/LED/RGB/parlaklık alanları UI'da gösterilir.
6. Kullanıcı switch, renk butonu veya slider değiştirirse JSON control komutu oluşturulup CONTROL_UUID characteristic'ine yazılır.
7. Bağlantı koparsa GATT kapatılır, UI bağlı değil durumuna alınır ve yeniden bağlanma denenir.

8. Wi-Fi/MQTT Sürümü

8.1 ESP32 Wi-Fi/MQTT tarafı

ESP32 Wi-Fi kodu WiFi, PubSubClient, ArduinoJson ve Adafruit_NeoPixel kütüphanelerini kullanır. ESP32 önce Wi-Fi ağına bağlanır, ardından MQTT broker'a bağlanarak telemetri yayınlar ve kontrol topic'ini dinler.

Sabit / Alan	Açıklama
WIFI_SSID / WIFI_PASSWORD	ESP32'nin bağlanacağı Wi-Fi bilgileri.
MQTT_HOST / MQTT_PORT	MQTT broker IP adresi ve portu. Varsayılan MQTT portu 1883 olarak kullanılmıştır.
DEVICE_ID	esp32-s3-001
TELEMETRY_TOPIC	iot/esp32-s3-001/telemetry
CONTROL_TOPIC	iot/esp32-s3-001/control
TELEMETRY_INTERVAL_MS	200 ms. ESP32 düzenli telemetri yayınlar.

ESP32'nin MQTT ile gönderdiği telemetri formatı:

```
{
  "deviceId": "esp32-s3-001",
  "gasValue": 1234,
  "ledOn": true,
  "red": 255,
  "green": 0,
  "blue": 0,
  "brightness": 255
}
```

ESP32'nin MQTT control topic üzerinden beklediği kontrol formatı:

```
{
  "ledOn": true,
  "red": 255,
  "green": 0,
  "blue": 0,
  "brightness": 255
}
```

Not: Wi-Fi kodunda `WiFi.setSleep(false)` kullanılmıştır. Bu, ESP32'nin Wi-Fi güç tasarrufundan kaynaklı gecikmelerini azaltmaya yardımcı olur.

9. Spring Boot Backend

Backend, Wi-Fi/MQTT sürümünde sistemin merkezi katmanıdır. ESP32'den gelen MQTT telemetrisini alır, son cihaz state'ini RAM'de tutar, cihaz bağlantı durumunu zaman aşımına göre belirler, Android'e WebSocket ile state gönderir ve geçmiş kayıtları SQLite'a yazar.

9.1 Backend bağımlılıkları

Bağımlılık	Kullanım Amacı
spring-boot-starter-web	REST endpoint oluşturmak için.
spring-boot-starter-jdbc	JdbcTemplate ile SQLite işlemleri için.
sqlite-jdbc	SQLite veritabanı sürücüsü.
spring-boot-starter-websocket	Android uygulama ile WebSocket iletişimi için.
org.eclipse.paho.client.mqttv3	Backend'in MQTT broker'a bağlanması, publish/subscribe yapması için.
jackson-databind	JSON parse ve JSON üretim işlemleri için.

9.2 Backend sınıfları

Sınıf	Görevi
Esp32BackendApplication	Spring Boot uygulamasının başlangıç sınıfı.
DeviceController	REST tarafında /api/device/history endpoint'ini sağlar.
DeviceService	Son state, bağlantı kontrolü, veri kaydı, MQTT control publish ve WebSocket broadcast işlemlerinin merkezidir.
MqttTelemetrySubscriber	MQTT telemetry topic'ine subscribe olur ve gelen payload'u DeviceService'e aktarır.
MqttPublisher	Android'den gelen kontrol komutlarını MQTT control topic'ine publish eder.
DeviceWebSocketHandler	Android WebSocket mesajlarını alır; control mesajını DeviceService'e yönlendirir; bağlantıda son state'i gönderir.
DeviceWebSocketBroadcaster	Bağlı tüm WebSocket session'larına state mesajı yayınlar.
WebSocketConfig	/ws/device endpoint'ini WebSocket handler ile eşler.
ControlRequest	LED açık/kapalı, RGB ve parlaklık komutu için DTO sınıfıdır.

9.3 REST ve WebSocket endpointleri

Endpoint	Tip	Açıklama
GET /api/device/history?limit=N	REST	SQLite telemetry_history tablosundan son N kaydı döndürür. Android geçmiş ekranı bunu kullanır.
/ws/device	WebSocket	Android uygulama bu adrese bağlanır; backend state mesajlarını anlık yollar ve control mesajlarını buradan alır.

9.4 Cihaz bağlantı durumu

Backend cihazın bağlı olup olmadığını ayrı bir “connected” paketiyle değil, son MQTT telemetri zamanına göre anlar. DeviceService içinde DEVICE_TIMEOUT_MS = 2000 olarak tanımlanmıştır. Son telemetri üzerinden 2 saniyeden fazla süre geçmişse cihaz bağlı değil kabul edilir.

Durum	Backend davranışı
Telemetri geliyor	lastTelemetryAtMs güncellenir, cihaz bağlı kabul edilir, son state Android’e broadcast edilir.
Telemetri 2 saniyeden uzun süre gelmezse	deviceConnected false olur; scheduled kontrol bunu fark edince Android’e bağlı değil state’i yollar.

10. Android Uygulama Yapısı

10.1 Wi-Fi Android uygulaması

Wi-Fi Android uygulaması, backend ile iki kanal üzerinden konuşur: anlık veriler için WebSocket, geçmiş kayıtlar için HTTP GET. MainActivity, UI state yönetimini yapar; DeviceWebSocketClient bağlantı ve mesaj yönetimini; DeviceApiClient ise geçmiş verilerini almayı sağlar.

Dosya	Görev
MainActivity.java	Arayüz bileşenlerini bağlar, WebSocket state mesajlarını UI’ya işler, kullanıcı kontrol değişikliklerini backend’e yollar, geçmiş verisini periyodik yükler.
AppConfig.java	BASE_URL, WS_URL, HISTORY_REFRESH_MS, CONTROL_DEBOUNCE_MS ve RECONNECT_DELAY_MS değerlerini tutar.
DeviceWebSocketClient.java	OkHttp WebSocket ile /ws/device bağlantısını kurar, state mesajlarını parse eder, control mesajı gönderir, kopunca yeniden bağlanır.
DeviceApiClient.java	GET /api/device/history?limit=N isteği atar ve JSON array’i HistoryItem listesine çevirir.
DeviceState.java	Backend state JSON’unu Android modeline dönüştürür.
ControlCommand.java	Android kontrol değerlerini type=control JSON formatına dönüştürür.
HistoryItem.java	Geçmiş kayıt JSON’unu model haline getirir.
HistoryFormatter.java	Geçmiş kayıtları TextView içinde tablo benzeri hizalı metne çevirir.

10.2 Android UI davranışı

- Cihaz bağlıysa bağlantı metni yeşil tonla “Bağlı” olarak gösterilir; cihaz bağlı değilse kırmızı tonla “Bağlı Değil” gösterilir.
- Gaz değeri, LED durumu, RGB değerleri, parlaklık ve son güncelleme zamanı ayrı TextView alanlarında gösterilir.
- LED switch, RGB seekbar’ları, parlaklık seekbar’ı ve renk preset butonları kullanıcıdan kontrol komutu üretir.
- Kontrol komutları 100 ms debounce ile gönderilir; bu sayede slider hızlı oynatılırken gereksiz çok sayıda komut gönderilmez.
- Son yerel kullanıcı değişikliğinden sonra kısa süre boyunca backend’den gelen state’in UI kontrollerini ezmesi engellenir.
- Geçmiş kayıtlar 30 saniyede bir yenilenir ve en fazla 20 satır formatlanarak gösterilir.

11. JSON Veri Formatları

Veri Tipi	Örnek	Açıklama
BLE state	<code>{"c":true,"g":1200,"l":true,"r":255,"gr":0,"b":0,"br":255}</code>	ESP32'den Android'e notification ile gider.
BLE control	<code>{"l":true,"r":255,"gr":0,"b":0,"br":255}</code>	Android'den ESP32 control characteristic'ine yazılır.
MQTT telemetry	<code>{"deviceId":"esp32-s3-001","gasValue":1200,"ledOn":true,"red":255,"green":0,"blue":0,"brightness":255}</code>	ESP32'den backend'e MQTT publish edilir.
MQTT control	<code>{"ledOn":true,"red":255,"green":0,"blue":0,"brightness":255}</code>	Backend'den ESP32'ye MQTT control topic üzerinden gider.
WebSocket state	<code>{"type":"state","deviceConnected":true,"gasValue":1200,...}</code>	Backend'den Android'e anlık cihaz state'i olarak gider.
WebSocket control	<code>{"type":"control","ledOn":true,"red":255,"green":0,"blue":0,"brightness":255}</code>	Android'den backend'e WebSocket ile gider.

12. Veritabanı Tasarımı

Backend, SQLite üzerinde telemetry_history tablosunu otomatik oluşturur. Tablo, gaz değeri ve LED state bilgisini geçmiş kayıt olarak saklamak için kullanılır.

Alan	Tip	Açıklama
id	INTEGER PRIMARY KEY AUTOINCREMENT	Kayıt kimliği.
recorded_at	TEXT NOT NULL	Kayıt zamanı. Europe/Istanbul saat diliminde yyyy-MM-dd HH:mm:ss formatı kullanılır.
gas_value	INTEGER NOT NULL	MQ-2 sensöründen gelen analog değer.
led_on	INTEGER NOT NULL	LED açık ise 1, kapalı ise 0.
red	INTEGER NOT NULL	Kırmızı renk değeri.
green	INTEGER NOT NULL	Yeşil renk değeri.
blue	INTEGER NOT NULL	Mavi renk değeri.
brightness	INTEGER NOT NULL	Parlaklık değeri.

Not: DeviceService içinde HISTORY_INTERVAL_MS = 60_000 olduğu için geçmiş kaydı yaklaşık dakikada bir alınır. Kod, aynı dakika içinde kayıt güncellemek yerine her uygun aralıkta yeni satır ekleyecek yapıdadır.

13. Kurulum ve Çalıştırma

13.1 ESP32 BLE sürümü

1. Arduino IDE'de ESP32 kart desteğinin kurulu olduğundan emin olun.
2. Adafruit_NeoPixel, ArduinoJson ve ESP32 BLE kütüphanelerinin kullanılabilir olduğundan emin olun.
3. esp32_bluetooth.ino dosyasını açın.
4. GAS_PIN ve WS2812_PIN değerlerinin kendi kartınıza uygun olduğunu kontrol edin.
5. Kodu ESP32-S3 karta yükleyin.
6. Android BLE uygulamasında TARGET_MAC_ADDRESS değerini kendi ESP32 BLE MAC adresinizle değiştirin.
7. Android uygulamayı telefona yükleyin ve Bluetooth izinlerini verin.
8. Uygulama açılınca ESP32'ye bağlanmalı, gaz değeri ve LED durumu görüntülenmelidir.

13.2 Wi-Fi/MQTT sürümü

1. MQTT broker'ın çalıştığından ve 1883 portunun açık olduğundan emin olun.
2. esp32_emb_src.ino içinde WIFI_SSID, WIFI_PASSWORD, MQTT_HOST, MQTT_USERNAME ve MQTT_PASSWORD alanlarını kendi ortamınıza göre değiştirin.
3. Backend application.properties içinde server.port, spring.datasource.url, mqtt.broker, mqtt.username, mqtt.password, telemetry topic ve control topic değerlerini düzenleyin.
4. esp32-backend klasöründe Maven ile backend'i çalıştırın: mvn spring-boot:run veya mvn clean package sonrası jar çalıştırma.
5. Android AppConfig.java içinde BASE_URL ve WS_URL değerlerini backend IP/port bilgisine göre değiştirin.
6. ESP32 kodunu karta yükleyin ve seri monitörde Wi-Fi/MQTT bağlantısının başarılı olduğunu kontrol edin.
7. Android uygulamayı çalıştırın; WebSocket bağlanmalı ve state mesajları geldikçe UI güncellenmelidir.

```
# Backend örnek çalıştırma
cd ESP32_Mobile_app_wifi/esp32-backend
mvn clean package
java -jar target/esp32-backend-0.0.1-SNAPSHOT.jar
```

14. Test Senaryoları

Test	Beklenen Sonuç
ESP32 açık, Android BLE uygulaması açık	Telefon ESP32'ye bağlanır, gaz değeri ve LED durumu 200 ms civarında güncellenir.
BLE bağlantısı kesilir	Android "Bağlı değil" durumuna geçer ve yeniden bağlanmayı dener.
LED switch açılır/kapılır	ESP32 WS2812 LED gerçek durumda açılır veya kapanır.
RGB slider değiştirilir	Renk önizlemesi ve fiziksel LED rengi güncellenir.
Wi-Fi sürümünde ESP32 MQTT broker'a bağlanır	Backend terminalinde MQTT telemetry geldi logları görülür.
Android Wi-Fi uygulaması WebSocket'e bağlanır	Bağlantı durumu bağlı görünür ve state verileri anlık gelir.
ESP32 kapatılır	Backend 2 saniye civarında cihazı bağlı değil kabul eder; Android UI bağlı değil olur.
Geçmiş endpoint çağrılır	/api/device/history?limit=20 son kayıtları JSON array olarak döndürür.

15. Hata Yönetimi ve Öneriler

15.1 Olası sorunlar

Sorun	Muhtemel Sebep	Çözüm Önerisi
Android BLE cihazı bulamıyor veya bağlanmıyor	MAC adresi yanlış, Bluetooth kapalı, izin eksik veya ESP32 advertise etmiyor.	TARGET_MAC_ADDRESS değerini kontrol edin, Bluetooth izinlerini verin, ESP32 seri monitörde BLE başladı çıktısını kontrol edin.
GATT hata mesajları sık geliyor	Bağlantı kopuyor, cihaz kapalı, eski GATT düzgün kapanmamış veya yeniden bağlanma döngüsü çalışıyor.	closeGatt sonrası yeniden bağlanma gecikmesini koruyun; cihaz açık değilse bağlantı denemesini sınırlayın.
Wi-Fi sürümünde Android bağlı değil görünüyor	Backend URL yanlış, WebSocket portu kapalı veya backend çalışmıyor.	AppConfig BASE_URL/WS_URL ve server.port değerlerini kontrol edin.
MQTT telemetry gelmiyor	Broker IP, kullanıcı adı/şifre veya topic yanlış.	ESP32 ve backend tarafındaki topic değerlerinin birebir aynı olduğundan emin olun.
Geçmiş kayıt oluşmuyor	ESP32 bağlı değil kabul ediliyor veya SQLite yolu yazılabilir değil.	spring.datasource.url yolunu ve dosya izinlerini kontrol edin.
LED rengi beklenenden farklı	WS2812 renk sırası veya pin farklı olabilir.	NEO_GRB ayarını, LED pinini ve kart pinout'unu kontrol edin.

15.2 İyileştirme önerileri

- BLE tarafında sabit MAC adresi yerine cihaz tarama ekranı eklenebilir. Bu, farklı ESP32 kartlarında uygulamayı daha pratik hale getirir.
- application.properties ve Arduino içindeki kullanıcı adı/şifre gibi bilgiler doğrudan repoda tutulmamalı; örnek config dosyası ve local secret dosyaları kullanılmalıdır.
- Wi-Fi sürümünde MQTT mesajları için QoS seviyesi ihtiyaçlara göre 1 yapılabilir; fakat gecikme ve ağ yükü etkisi test edilmelidir.
- SQLite geçmiş kaydı için aynı dakika içinde update, dakika değişince insert mantığı isteniyorsa DeviceService saveHistory bölümü minute bucket mantığına göre geliştirilebilir.
- Android Wi-Fi uygulamasında bağlantı durumu yalnızca WebSocket bağlantısı değil, backend'in deviceConnected alanı ile birlikte yorumlanmalıdır.
- Backend için basit bir health endpoint eklenebilir: GET /api/health.
- MQ-2 sensörü için kalibrasyon, eşik değer ve alarm modu eklenerek proje daha gerçekçi bir gaz alarm sistemine dönüştürülebilir.
- Kodlar class yapısı bakımından daha da ayrıştırılabilir: ESP32 tarafında LedController, GasSensorReader, MqttManager gibi sınıflar kullanılabilir.

16. Sonuç

Bu proje, ESP32 tabanlı bir IoT projesinin iki farklı haberleşme yaklaşımıyla nasıl geliştirilebileceğini göstermektedir. BLE sürümü daha sade ve doğrudan bir telefon-cihaz bağlantısı sağlarken, Wi-Fi/MQTT sürümü backend, veritabanı ve WebSocket desteği sayesinde daha genişletilebilir bir mimari sunar. Projede sensör okuma, gerçek zamanlı mobil arayüz, LED kontrolü, bağlantı durum takibi, MQTT mesajlaşması, WebSocket yayını ve SQLite geçmiş kaydı gibi birçok temel IoT konusu birlikte uygulanmıştır.

Ek A – Kod Dosyaları ve İçeriği

Dosya	Dokümantasyonda Kullanılan İçerik
README.md	Projenin genel amacı, BLE ve Wi-Fi yaklaşımı, donanım bileşenleri ve repo fikri.
ESP32_Mobile_app_bluetooth/esp32_bluetooth/esp32_bluetooth.ino	BLE service/characteristic yapısı, 200 ms state bildirimi, LED ve gaz sensörü mantığı.
ESP32_Mobile_app_bluetooth/.../MainActivity.java	Android BLE GATT bağlantısı, notification açma, MTU, debounce ve UI güncelleme akışı.
ESP32_Mobile_app_wifi/esp32_emb_src/esp32_emb_src.ino	Wi-Fi bağlantısı, MQTT telemetry/control topicleri, ESP32 LED ve gaz sensörü yönetimi.
ESP32_Mobile_app_wifi/esp32-backend/.../DeviceService.java	Backend state yönetimi, bağlantı timeout, SQLite history ve MQTT control publish mantığı.
ESP32_Mobile_app_wifi/esp32-backend/.../MqttTelemetrySubscriber.java	MQTT telemetry subscribe ve payload'un DeviceService'e aktarılması.
ESP32_Mobile_app_wifi/esp32-backend/.../MqttPublisher.java	Backend'den ESP32'ye MQTT control mesajı gönderilmesi.
ESP32_Mobile_app_wifi/esp32-backend/.../DeviceWebSocketHandler.java	Android WebSocket bağlantısı ve control/state mesaj akışı.
ESP32_Mobile_app_wifi/ESP32_Mobile/.../DeviceWebSocketClient.java	Android WebSocket client, reconnect ve state parse mantığı.
ESP32_Mobile_app_wifi/ESP32_Mobile/.../DeviceApiClient.java	Android geçmiş kayıtlarının REST API üzerinden alınması.